Operating Systems

6CM503

Computer GE 645 with OS Multix and its creators around

Laboratory and home training 2025/26

Incremental learning for Assessments 01 - 04

Dmitry A. Zaitsev

Module Leader

- Dr. Dmitry A. Zaitsev
- Email: d.zaitsev@derby.ac.uk, Phone : 01332592678
- Office: MS229. Recommended days for individual communication: Monday, Wednesday.
- Website: https://www.derby.ac.uk/staff/dmitry-zaitsev

Purpose

Laboratory and home training pursues the main purpose of incremental learning for smooth and successful implementation of assessments. It is divided into sections after assessments 01 – 04 and refers to the corresponding Assessment Brief detailing the job recommended to do each week for successful and timely completion of the assessment.

1. Laboratory and home training for Assessment 1 – Linux Command Line Interface (CLI): Shell Script Design

We start with CLI training, learning commands and their options. Then, following individual tasks of Assessment Brief 01, we compose a single-line script in the form of a pipe and finally design a system script for monitoring OS resources.

1.1. Recommended schedule of assessment implementation

Week no.	What to do	Outcome
1	Register with Azure cloud. Start working with Linux VM via RDP using GUI following pattern of Lecture 1.	Azure cloud VM account. Screen images of using Linux GUI and basic tools.
2	Start working with Linux VM via SSH using CLI following the pattern of Lecture 2. Complete Linux CLI training sequences for bash, A. Files and directories	Sequence of bash commands to implement Linux CLI training sequences for bash, A. Files and directories
3	Complete Linux CLI training sequences for bash, B. Protection of files, C. Processes. Start composing the pipe, select required commands and their options.	Sequence of bash commands to implement Linux CLI training sequences for bash, B. Protection of files, C. Processes. List of commands required to compose the pipe.
4	Complete Linux CLI training sequences for bash, D. Filters: sort, E. Filters: grep. Complete composing and testing the pipe.	Sequence of bash commands to implement Complete Linux CLI training sequences for bash, D. Filters: sort, E. Filters: grep. The pipe (a single line script) file and results of its testing.
5	Start composing a script. Select required commands and their options. Start adding control flow and run	List of commands required to compose the pipe and their

	scrip drafts.	required options. Script draft.
6	Complete composing and testing the script. Compose and submit the assessment report.	The script file and results of its testing. The assessment report and attachments.

1.2. Linux CLI training sequences for bash

A. Files and directories

1. Create the following structure of directories and files. Give Your name to the top directory. This directory will be your home directory.

```
                [Your name]
                     !
        vespa        !     vaz volvo
    ford daf         !     citroen
                 Mashini
            _________!_________
            !                  !
    Razni                Francuz
```

2. Stay in Your home directory.
3. Move *ford* to directory *Mashini*.
4. Move all files, which names begin from “v” to directory *Razni*.
5. Move *vaz* to directory *Mashini*.
6. Stay in directory *Mashini*.
7. Copy *daf* to this directory.
8. Stay in directory *Raznie*.
9. Move *vespa* to directory *Francuz* and rename it into *vespa2*.
10. Create three files with arbitrary names.
11. Go to directory *Mashini*.
12. Copy two files from *Raznie* to *Francuz* using only one command.
13. Stay in Your home directory.
14. Show on the screen all levels under Your home directory.

B. Protection of files

1. Stay in Your home directory and execute the command

ls -al > LIST

which creates a file *LIST* showing initial protection of filers.

2. Let files *.profile* and *fruits* and directory *practice* are situated in Your home directory. They will be used further.

3. Change protection of file *.profile* that no other category of users except you can read the file content. Check the result.

4. Change protection of file fruits that only You or members of Your group can read file and write to it but all other users cannot do it.

5. Check which group files of Your directory belongs to. Check which group you belong to. Change using a command the group of a file. Check in file */etc/group* who belongs and who does not belong to Your group.

6. Cancel all access rights for all categories of users for directory practice. Execute command ls - al practice. Cancel Your own x-right. Execute command ls - al practice.

C. Processes

1. Determine who works in system and what processes are started by each user. For each process determine its state and priority. Find processes unattached to no one terminal.

2. Try to delete processes started by other users.

3. Create a command *dummy* which incessantly prints into standard output stream its name. Start the command. Interrupt its running and delete the process.

4. Start process *dummy* in background mode. Redirect its output into a file. Find the corresponding process in the processes list. Determine its state. Delete it.

5. Do the same operations decreasing the process priority using command *nice*. Change priority of running process using command *renice*.

6. Start a few copies of the process *dymmy* in background mode; use redirection of output into a dummy device. Find started processes in the processes list. Exit from the system. Enter the system. Check the processes list.

7. Do the same operations starting processes in uninterrupted mode *nohup*. Compare process states after repeated entering into the system. Delete started processes.

8. Enter commands of starting three dummy processes in various instances of time (for example, after 3, 5, and 7 minutes). Watch starting the processes then delete them. Do processes start after a user exits the system?

D. Filters: sort

1. Create a file named *family* that contains surnames, names and birth dates of a few persons. Each line has the following form:

smith john 3/1/50

Separate name and birth date with a tabulation (<tab>).

2. Use command *sort* to sort the file in alphabetic order.

3. Use an option (key) of command *sort* to sort the file in reverse order.

4. Create another file named *work* which contains surnames, names and birth dates of a few colleagues (group mates) having the same format as file family. Include Your personal data into the file.

5. Sort both files simultaneously by command *sort*. Pay your attention that your name is printed twice. Use an option (key) of command *sort* to print your name only once.

6. Create sorted files from files *work* and *family*. Use the key "–o" or redirection arrow ">". If You use ">" please choose a new file name for sorted file.

7. Try using the key “-m” of command sort to sort source files work and family. Then try to sort with the ket “-m” already sorted files. Pay attention that this key does not sort files but merges sorted files.

8. Sort Your files by the alphabetic order of name.

9. If a few persons have the same surname the program do not sort them automatically according to names. To overcome it, insert the key “-2” which enlarges sorting field till the column number 3.

10. Sort files in the order of the birth month. Try it with the key “+2”. Look how it has been done for people born in October, November, December. Try doing it with the key “+2n”.

E. Filters: grep

1. Create a file named price.veg with the following data

Lettuce	.89
tomatoes	0.89
broccoli	.79
cauliflower	0.89
parsely	2.14
avocado	1.19
carrots	0.69
ctltry	0.59
string-beans	1.29
onions	0.29
asparagus	1.39
corn	0.85
cabbage	0.99
lima-beans	1.29
etllow-beans	1.17
mushrooms	1.09
peppers	1.39
potatoes	0.39
artichokes	1.59
black-beans	1.89
blacheyed-beans	1.49

2. Sort the file twice: once in alphabetic order and twice according to prices. Write results into two new files.

3. Separate all beans in a file.

4. Find vegetables which names start with letter “s”.

5. Find vegetables which names start with a vowel.

6. Find vegetables which price ends with digit “9”.

7. Find vegetables which price ends with digits “09”, “39”, or “79”.

8. Find vegetables which price does not end with digit “9”.

9. Find vegetables which names do not start with letters from “a” to “m”.

10. Find vegetables which price ends with digit “9” but does not end with “39”, “59”, “79”, and “89”.

1.3. Recommendations for the pipe design

- At first, execute required commands manually one-by-one.
- Then compose selected commands into a pipe.
- Save the pipe in a file.
- Give to the file permission to execute it.
- Use *cut* to cut vertical columns.
- Use *grep*, *head*, *tail* to manipulate rows.
- Count specified elements with *wc*.

Basic items:

1. Beginning/finishing work with system. Users, groups.
2. Files and directories. Protection of files (directories).
3. Process control.
4. Editing text.
5. Redirection of input/output. Pipes, filters.

Basic commands of Unix:

- ✓ Users and groups: *login, exit, who, newgrp, man*.
- ✓ Files and directories: *ls, cat, cp, mv, rm, pwd, cd, mkdir, rmdir, find (file, ln, diff, comm, cmp)*.
- ✓ Protection: *chmod, chown, chgrp, umask*.
- ✓ Editing files: *ed, vi*.
- ✓ Processes: *ps, &, kill, nice, at, nohup, renice*.
- ✓ Filters: *wc, sort, grep, sed, cut, paste, pr (awk)*.

1.4. Recommendations for the script design

- Implement tracking via periodic (after 1-5 minutes) repeating of certain actions and comparing the current state with the previous state saved in an auxiliary temporary file.
- For periodic start, use either command *sleep* or *at*.
- Create auxiliary temporary files in directory */tmp*; provide unique names of temporary files (using built-in variable *$$*).
- Provide correct completion of the process (cleaning temporary files, etc.) on its deletion via processing of corresponding signals.
- The source of additional information – man-pages of shell (bash) and commands.

Basic items:

1. Structure of *shell* command line, interpretation of meta-symbols, patterns.

2. Variables of *shell*, built-in variables.
3. Arguments and options of commands; return codes; substitution of results of a command execution.
4. Control of operations sequences: branching and lops. Checking conditions.
5. Subroutines and signals processing in shell scripts.

Basic constructs of Shell language:

Meta-symbols:

> >> < <<s <<\s <<'s' |
. * ? [abc] [d-f]
; (command) {commands} & command1&&command2 command1||command2
'string' "string" `command`

Variables and parameters:

variable=value read variable echo $variable
$variable ${variable}
${variable-value} ${variable=value}
${variable?string} ${variable+value}
$1 $2 etc
shift
set argument1 argument2 etc

2. Laboratory and home training for Assessment 2 – Linux System Calls: Client-Server Design using Inter-Process Communication (IPC)

We compose a conventional single-process utility program according to the individual tasks of Assessment Brief 02, debug, and test it. Then we run example programs for IPC, learning how clients and servers interact. Finally, following the student's individual task of Assessment Brief 02, we compose a client-server program for a given IPC (according to the individual task), debug and test it, focusing on getting evidence that the corresponding IPC facilities were created within the Linux kernel, and compose graphs of client-server interaction.

Actually, we just split a utility program in two applications: a client – for inputting and outputting information – and a server – for the implementation of the information processing function. To connect a client with a server, we use Linux IPC facilities given in your individual task.

2.1. Recommended schedule of assessment implementation

Week no.	What to do	Outcome
7	Design the utility program in C language and test it. Implement basic data processing as a function of C language.	Utility program, source C code and results of its testing.
8	Run and test example programs of client and server communication via IPC. Collect evidence of the corresponding IPC created within the Linux kernel. Compose schemes of interaction between processes.	Command lines and interaction listing for communicating processes. Listing of the created IPC state within the

		Linux kernel. Images of screens. Schemes of client-server interaction inscribed with system calls.
9	Design client and server prototypes to provide service using IPC for the individual task; use the basic function from the utility program	Source C files of the client and server prototypes. Brief description of prototype programs.
10	Design client and server to provide service using IPC for the individual task refining early developed prototypes; use the basic function from the utility program	Source C files of the client and server. Brief description of programs.
11	Test developed client-server pairs. Collect evidence of the corresponding IPC created within the Linux kernel. Compose schemes of interaction within client-server pairs inscribed with system calls.	Command lines and interaction listing for communicating processes. Listing of the created IPC specification within the Linux kernel. Images of screens. Schemes of client-server interaction inscribed with system calls.
12	Compose and submit the assessment report.	The assessment report and attachments.

2.2. Recommendations for utility program design

Compose a utility program with command-line interface to provide the required service, and implement at least 3 options (keys). Organize processing of information required by the individual task as a C language function. Work with arrays of characters and pointers directly. Use neither standard libc functions for string processing nor other libraries.

Plot: In the following tasks, we will be using the same basic code for required service (C language function) in the context of IPC and the kernel loadable module.

Recommendations:

- Discuss with teacher and decide on the utility options.
- Run the example program and study how command line interface is implemented.
- Compose a function for the required processing of information.
- Compose the main program to implement the command line interface and the function call.
- Debug and test the program.
- Create and consider intermediate codes: after preprocessor, assembler, compiler (object code).

2.3. References to IPC examples

Pipes: creation of processes *fork_ex.c*; communication via pipes *pipe_ex.c*

Messages: communication via messages *mq_client.c, mq_server.c, mq_common.c*

Semaphores: synchronization of dining philosophers via semaphores *ph.c, super_ph.c*; using shared segments of memory *seg_ex.c, shm_client.c, shm_server.c*

Sockets for datagram communication: communication via IP-UDP sockets *udp_client.c, udp_server.c*

Sockets for stream communication: communication via IP-TCP sockets: tcp_client.c and tcp_server.c

2.4. Recommendations on IPC implementation

6.3.1. IPC via pipes

Implement the required information processing using a pair of specialized processes – client and server; provide the information communication between the processes via Linux pipes.

Examples: creation of processes *fork_ex.c*; communication via pipes *pipe_ex.c*

Recommendations:

- Use the utility program designed according to your individual task on labs.
- Compose a client that periodically inputs data and sends it to server.
- Compose a server that receives data from client, processes it, and returns the result back to client.
- Debug and test the programs.
- Find out commands required to observe the corresponding pipes created within the Linux kernel and collect evidence of actual using pipes.

6.3.2. IPC via messages

Implement the required information processing using a pair of specialized processes – client and server; provide the information communication between the processes via Linux messages.

Example: communication via messages *mq_client.c, mq_server.c, mq_common.c*

Recommendations:

- Use the utility program designed according to your individual task on labs.
- Compose a client that periodically inputs data and sends it to server.
- Compose a server that receives data from client, processes it, and returns the result back to client.
- Debug and test the programs.
- Find out commands required to observe the corresponding message queues created within the Linux kernel and collect evidence of actual using messages.

6.3.3. IPC via semaphores

Implement the required information processing using a pair of specialized processes – client and server; provide the information communication between the processes via Linux semaphores and shared segments of memory.

Example: synchronization of dining philosophers via semaphores *ph.c, super_ph.c*; using shared segments of memory *seg_ex.c, shm_client.c, shm_server.c*

Directions:

- Use the designed utility program according to your individual task.
- Compose a client that periodically inputs data and sends it to server.
- Compose a server that receives data from client, processes it, and returns the result back to client.
- Debug and test the programs.
- Find out commands required to observe the corresponding semaphores and shared segments of memory created within the Linux kernel and collect evidence of actual using semaphores and shared segments of memory.

6.3.5. IPC via IP-UDP sockets

Implement the required information processing using a pair of specialized processes – client and server; provide the information communication between the processes via Linux IP-UDP sockets.

Example: communication via IP-UDP sockets *udp_client.c, udp_server.c*

Directions:

- Use the designed utility program according to your individual task.
- Compose a client that periodically inputs data and sends it to server.
- Compose a server that receives data from client, processes it, and returns the result back to client.
- Use coordinated identification of client and server with a socket – IP-address and port number.
- Debug and test the programs.
- Try connection between remote computers.
- Find out commands required to observe the corresponding sockets created within the Linux kernel and collect evidence of actual using sockets.

3. Laboratory and home training for Assessment 3 – Linux Application Design using Source Code Libraries and Containers

Building professional open-source systems represents a certain challenge to a programmer, requiring integration with the operating system environment and using system tools.

At first, we download LAPACK and required additional components such as BLAS from a repository. Then we configure LAPACK, modifying the corresponding configuration files and install required tools, for instance, the FORTRAN compiler. We build the LAPACK library and its C language interface LAPACKE, run example tests, and benchmark our computer using LAPACK for solving big random dense systems of linear algebraic equations, representing benchmarks in vivid graphical form. We

compose our own application to resolve a given linear system, which is obtained from a string containing a student surname and name.

Running many application systems on a multi-user server computer represents a serious challenge for the system integrity. Different applications require a series of third-party components (libraries), often having different versions for the same component, while simply copying third-party components violates license agreements.

The Docker toolset resolves the situation, allowing us to integrate in a container a specific operational environment required by the application system, including a specific operating system and libraries. We install Docker, and compose and benchmark containers with LAPACK and with the individual test application on Linux, MacOS, and Windows platforms that allow us to evaluate the efficiency of the approach.

3.1. Recommended schedule of assessment implementation

Week no.	What to do	Outcome
13	Download and install LAPACK. Run LAPACKE example applications for solving dense systems of linear algebraic equations.	LAPACK, BLAS, LAPACKE installed and tested.
14	Benchmark your VM using LAPACK, compare obtained results with theoretical evaluations of computational complexity.	Benchmarks of solving big systems collected in tabular and graphical form.
15	Design a program for calling LAPACK functions for solving a given system of equations according to the student individual task, check obtained results.	C program code for solving a given linear system, check of obtained results.
16	Install and test Docker with standard containers. Run Docker on Linux, Window, and MacOS.	Docker installed and tested.
17	Design a series of Docker containers for running LAPACK and your program that calls LAPACK functions.	Docker containers for running LAPACK and application that uses LAPACK.
18	Benchmark LAPACK containers on Linux, Window, and MacOS. Compose and submit the assessment report.	Benchmarks of solving big systems using Docker containers, collected in tabular and graphical form. The assessment report and attachments.

3.2. Instructions on installing and testing LAPACK

```
# install basic tools
sudo apt update
sudo apt install build-essential
sudo apt install gfortran

# download LAPACK
wget https://github.com/Reference-
LAPACK/lapack/archive/refs/tags/v3.12.1.tar.gz
ls
```

```
tar -zxvf v3.12.1.tar.gz
ls

# configure LAPACK
cd lapack-3.12.0
cp make.inc.example make.inc
nano make.inc

# build LAPACK libraries
ulimit -s unlimited
make
ls
cd CBLAS
ls
make
cd ..
cd LAPACKE
make
ls

# build LAPACK examples
cd example
make

# run LAPACK examples
./xexample_DGESV_colmajor
./xexample_DGESV_rowmajor -n 10
./xexample_DGESV_rowmajor -n 100
time ./xexample_DGESV_rowmajor -n 2000 > /dev/null
```

3.3. Example application (for a system considered at Lecture 13).

```
# Create file lp_ex1.c
# nano lp_ex1.c

/* solving the matrix equation A*x=b using LAPACK function
lapack_int LAPACKE_sgesv( int matrix_layout, lapack_int n,
lapack_int nrh, float* a, lapack_int lda, lapack_int* ipiv, float*
b,lapack_int ldb ); */

#include <stdio.h>
#include <lapacke.h>
#define size 3 /* dimension of matrix */
int main()
{
   lapack_int n=size, nrhs=1, lda=size, ldb=1, info, ipiv[size];
   float A[size*size] = {2.0,5.0,8.0,
                         3.0,-6.0,9.0,
                         4.0,7.0,-1.0};
   float b[size] = {13.0,25.0,-7.0};
   int i,j;
```

```
   info = LAPACKE_sgesv(LAPACK_ROW_MAJOR, n, nrhs, A, lda, ipiv, b,
ldb);

   for (i=0; i<n; i++) printf("%f\n", b[i]); /* print vector x */
}

# Create file Makefile
# nano Makefile

TOPSRCDIR = ../..
include $(TOPSRCDIR)/make.inc

.SUFFIXES: .c .o
.c.o:
     $(CC) $(CFLAGS) -I. -I../include -c -o $@ $<

.PHONY: all
all: lp_ex1

LIBRARIES = $(LAPACKELIB) $(LAPACKLIB) $(BLASLIB)

# Double Precision Examples
lp_ex1: lp_ex1.o $(LIBRARIES)
     $(FC) $(FFLAGS) $(LDFLAGS) -o $@ $^
     ./$@

.PHONY: clean cleanobj cleanexe
clean: cleanobj cleanexe
cleanobj:
     rm -f *.o
cleanexe:
     rm -f x*

# make and run the example
Make

# run the example
./lp_ex1
```

3.4. How to install and use Docker

```
# install Docker
sudo apt-get update
sudo apt-get remove docker docker-engine docker.io
sudo apt install docker.io
sudo systemctl start docker
sudo systemctl enable docker
docker --version
sudo groupadd docker
sudo usermod -aG docker $USER
sudo chmod 666 /var/run/docker.sock
```

```
# build and run simple containers
docker run hello-world
docker run -i -t ubuntu /bin/bash
docker run ubuntu /bin/echo Hello world
docker search ubuntu
docker ps -a
docker images

# build containers
mkdir myubuntu1
mkdir myubuntu2
mkdir myubuntu3
mkdir myubuntu4
ls
# build myubu1 – ubuntu with compilers
cd myubuntu1
nano Dockerfile
ls
docker image build -t myubu1 .
docker run -it myubu1
# build myubu2 – ubuntu with LAPACK
cd ../myubuntu2
nano Dockerfile
ls
cp ../../v3.12.1.tar.gz .
ls
docker image build -t myubu2 .
docker run -it myubu2
# build myubu3 – ubuntu with LAPACK, CBLAS, LAPACKE
docker image build -t myubu3 .
docker run -it myubu3
# build myubu3 – ubuntu with LAPACK, CBLAS, LAPACKE and my app
docker image build -t myubu4 .
docker run myubu4
```

3.5. Collection of Dockerfiles for incremental building and testing containers

```
# Dockerfile for myubu1
FROM ubuntu:latest
RUN apt-get update && apt-get install -y \
 build-essential \
 gfortran \
 python3
RUN /bin/bash

# Dockerfile for myubu2
FROM myubu1:latest
COPY v3.12.1.tar.gz .
RUN tar -zxvf v3.12.1.tar.gz
RUN cp /lapack-3.12.1/make.inc.example /lapack-3.12.1/make.inc
RUN ulimit -s unlimited && cd /lapack-3.12.1 && make
```

```
RUN /bin/bash

# Dockerfile for myubu3
FROM myubu2:latest
RUN cd /lapack-3.12.1/CBLAS && make && cd /lapack-3.12.1/LAPACKE &&
make && cd /lapack-3.12.1/LAPACKE/example && make
RUN /bin/bash

# Dockerfile for myubu4
FROM myubu3:latest
RUN mkdir /lapack-3.12.1/LAPACKE/myex
COPY lp_ex1.c /lapack-3.12.1/LAPACKE/myex
COPY Makefile /lapack-3.12.1/LAPACKE/myex
RUN cd /lapack-3.12.1/LAPACKE/myex && make
CMD /lapack-3.12.1/LAPACKE/myex/lp_ex1
```

4. Laboratory and home training for Assessment 4 – Linux Kernel Loadable Module Design

The problem of an optimal OS kernel influences performance of computers worldwide leading to carbon emission wasting electric power for keeping rarely used or even unused code within the kernel. In the present assessment we address this problem building a new kernel with a given set of configuration parameters and the extend the kernel functionality developing a kernel loadable module according to the student individual task.

The assessment involves incremental learning approach facilitating better student experience. We start with building Linux kernel from source code, booting from the built kernel and testing it applications developed in the previous assessments. That provides continuity of learning, significant for integrity of acquired knowledge and skills. Building the central component of an operation system such as kernel represents a special kind of work and students are encouraged to understand parameters of kernel which control the building process with their individual sets of parameters. In industrial systems, compiling optimized kernels considerably enhances efficiency and finally leads to low carbon emission saving electrical power. Actually, from building own kernel the process of operating system design starts.

Most frequent and significant functions of an operating system should be implemented within its kernel. Rebuilding and modifying kernel does not facilitate the system reliability. For flexible restricted modifications of kernel, Linux offers a concept of a kernel loadable module. During the process of its design, a loadable module uses files specifying symbols and other elements of the kernel environment left from building the current running kernel. Using the kernel environment information, we design an individual loadable module in a rather autonomous way and test its functionality after loading the module into the running kernel. This rather sophisticated system job is supported by teamwork on the same task with similar system interaction patterns using code of different individual functions developed during assessment 02.

4.1. Recommended schedule of assessment implementation

Week no.	What to do	Outcome

19	Install Linux kernel sources and required libraries and tools. Build, boot, and test a kernel with default configuration parameters.	Linux kernel sources installed. A new kernel built and tested.
20	Choose the kernel configuration parameters according to the student individual task. Build, boot, and test a kernel with default configuration parameters.	A new kernel with given configuration parameters built and tested.
21	Build, run, and test example leadable kernel module that implements solving of quadratic equation as a new system call.	Example kernel loadable module run and tested.
22	Design a kernel module that implements a new system call based on the utility program individual task	Kernel loadable module that solves individual task designed
23	Design a wrapper function for a new system call. Test kernel loadable module using the wrapper function.	Wrapper function to implement a new system call for using the designed loadable module. Loadable module tested.
24	Compose and submit the assessment report.	The assessment report and attachments.

4.2. Instructions on building Linux kernel (based on https://kernelnewbies.org/KernelBuild)

```
# Install required tools: git, make, gcc, libssl-dev and
(optionally) ctags, cscope, and/or ncurses-dev.

sudo apt-get install libncurses5-dev gcc make git exuberant-ctags bc
libssl-dev

# Download the latest stable tree

git clone
git://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-
stable.git

cd linux-stable

# Find the latest stable kernel tag by running

git tag -l | less

# Checkout the code associated with the kernel

git checkout -b stable tag

# Download the latest -rc tree

git clone
git://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git

cd linux

# Set up your kernel configuration
```

```
# Duplicate your current config
cp /boot/config-`uname -r`* .config
# Make the default config
make defconfig
# Change your config
make menuconfig
# Build the kernel (make -j5)
make
# Install the kernel
sudo make modules_install install
# Run your kernel
sudo update-grub2
# Additional useful commands
# Print or control the kernel ring buffer
dmesg
# see which drivers are used for each device
lspci -k
```

6.3. Individual tasks for Linux kernel requirements

Task/ Kernel feature	EXT4 File system Support	IPv6 Networking	Loadable Module Support	USB Device Support	Virtualization (KVM)	Security Module (SELinux)
1	+	–	+	–	+	–
2	+	+	–	–	–	+
3	–	+	–	+	–	+
4	–	–	+	+	+	–
5	–	–	–	+	+	+
6	+	+	+	–	–	–
7	–	–	+	+	–	+
8	+	+	–	–	+	–
9	–	+	+	–	+	–
10	+	–	–	+	–	+
11	–	+	+	+	–	–
12	+	–	–	–	+	+

6.4. Instructions on working with kernel loadable modules

```
# Get info on current kernel release
uname -r
# Show what kernel modules are currently loaded
lsmod
# Show information about a module
modinfo module_name
# List the options that are set for a loaded module with systool
from sysfsutils:
systool -v -m module_name
# Display the comprehensive configuration of all the modules
modprobe -c | less
# Display the configuration of a particular module
modprobe -c | grep module_name
# List the dependencies of a module (or alias), including the module
itself
modprobe --show-depends module_name
# Load a module
modprobe module_name
# Load a module by a file name—one that is not installed in the
/usr/lib/modules/kernel_release/
insmod file_name module_options
modprobe file_name
# Unload—remove—a module (any of)
rmmod module_name
modprobe -r module_name
modprobe --remove module_name
# Set module options
modprobe module_name parameter_name=parameter_value
```

6.5. Instruction on building Linux kernel loadable module (based on *https://docs.kernel.org/kbuild/modules.html*)

"kbuild" is the build system used by the Linux kernel. Modules must use kbuild to stay compatible with changes in the build infrastructure and to pick up the right flags to the compiler. Functionality for building modules both in-tree and out-of-tree is provided. The method for building either is similar, and all modules are initially developed and built out-of-tree.

To build external modules, you must have a prebuilt kernel available that contains the configuration and header files used in the build. Also, the kernel must have been built with modules enabled. If

you are using a distribution kernel, there will be a package for the kernel you are running provided by your distribution.

An alternative is to use the “make” target “modules_prepare.” This will make sure the kernel contains the information required. The target exists solely as a simple way to prepare a kernel source tree for building external modules.

```
# Build an external module
make -C <path_to_kernel_dir> M=$PWD
# Build against the running kernel
make -C /lib/modules/`uname -r`/build M=$PWD
# To install the module(s) just built, add the target
“modules_install” to the command
make -C /lib/modules/`uname -r`/build M=$PWD modules_install
# Starting from Linux 6.13, you can use the -f option instead of -C.
# This will avoid unnecessary change of the working directory.
# The external module will be output to the directory where you
invoke make
make -f /lib/modules/uname -r/build/Makefile M=$PWD
```

Options

($KDIR refers to the path of the kernel source directory, or the path of the kernel output directory if the kernel was built in a separate build directory.)

You can optionally pass MO= option if you want to build the modules in a separate directory.

make -C $KDIR M=$PWD [MO=$BUILD_DIR]

-C $KDIR

The directory that contains the kernel and relevant build artifacts used for building an external module. “make” will actually change to the specified directory when executing and will change back when finished.

M=$PWD

Informs kbuild that an external module is being built. The value given to “M” is the absolute path of the directory where the external module (kbuild file) is located.

MO=$BUILD_DIR

Specifies a separate output directory for the external module.

Targets

When building an external module, only a subset of the “make” targets are available.

make -C $KDIR M=$PWD [target]

The default will build the module(s) located in the current directory, so a target does not need to be specified. All output files will also be generated in this directory. No attempts are made to update the kernel source, and it is a precondition that a successful “make” has been executed for the kernel.

modules

> The default target for external modules. It has the same functionality as if no target was specified. See description above.

modules_install

> Install the external module(s). The default location is /lib/modules/<kernel_release>/updates/, but a prefix may be added with INSTALL_MOD_PATH (discussed in section Module Installation).

clean

> Remove all generated files in the module directory only.

help

> List the available targets for external modules.

○